

Practical 1: Responsive Ecommerce Admin Dashboard

1. What we have made (The "Elevator Pitch")

"Sir/Ma'am, I have built a **Responsive Admin Panel** for an Ecommerce platform. It serves as a central control room where a manager can see a snapshot of their business. It includes a navigation bar, a sidebar for menu options, and a main content area featuring **statistical cards** (for things like Total Sales, Orders, and Customers) and a **data table** to display recent product entries."

2. How we have built it

We followed a "Mobile-First" approach using the **Bootstrap Framework**:

- **Structure:** We used semantic HTML5 tags like `<nav>` for the top bar and `<main>` for the dashboard content.
- **The Grid:** The core layout uses the **Bootstrap Grid System**. We wrapped elements in `.container`, then `.row`, and finally `.col-*` classes.
- **Styling:** Instead of writing hundreds of lines of custom CSS, we used Bootstrap's utility classes (like `bg-primary` for blue backgrounds or `p-3` for padding) to keep the design clean and consistent.
- **Responsiveness:** We used **Media Queries** (built into Bootstrap) so that on a desktop, cards appear side-by-side (e.g., `col-md-3`), but on a mobile phone, they stack vertically (`col-12`).

3. The Basics (What you must understand)

- **The 12-Column Rule:** Bootstrap sees every row as 12 equal slices. If you want 3 equal boxes, each box must be `col-4` ($4 \times 3 = 12$).
- **Classes:** Classes are like "labels" we put on HTML tags to give them instant styling.
- **Breakpoints:** These are the screen sizes where the layout changes. Common ones are `sm` (small), `md` (medium), and `lg` (large).

4. What changes the examiner might ask you to make

Be ready for these "Live Tasks":

- **"Change the color of the cards":** You would look for classes like `bg-primary` (blue) and change them to `bg-success` (green) or `bg-danger` (red).

- **"Make the 4 cards appear as 2 cards per row on tablets":** You would change col-md-3 to col-md-6.
- **"Add a new item to the Sidebar":** You would go to the (unordered list) in the sidebar and copy-paste an (list item) tag.

5. Usual Viva Questions & Answers

- **Q: What is the difference between .container and .container-fluid?**
 - **A:** .container has a fixed width and leaves margins on the sides of the screen. .container-fluid is 100% width, stretching to the very edges of the screen.
- **Q: Why do we place the Bootstrap <link> in the <head> but the <script> tag at the bottom of the <body>?**
 - **A:** We put CSS in the head so the styles load before the user sees the page. We put JS at the bottom so the page structure loads first, preventing the website from feeling slow or "hanging" while scripts load.
- **Q: What is a "CDN" (Content Delivery Network)?**
 - **A:** It's a way to load Bootstrap from a fast external server instead of hosting the file on your own pendrive. (Note: In an exam, it's safer to have the file locally in case there's no internet!)
- **Q: What does the z-index property do in CSS?**
 - **A:** It controls the "stacking order." An element with a higher z-index will appear "on top" of an element with a lower one (useful for fixed navbars).

Practical 2: Exam Admin Dashboard

(Focus: HTML, CSS, Bootstrap)

1. What we have made

"Sir/Ma'am, for this practical, I have designed an **Exam Admin Dashboard**. While the layout is similar to the Ecommerce dashboard, the **data and functionality** are tailored for educational management. It allows an administrator to monitor active exams, view student attendance percentages, manage question banks, and see recent results. It's a specialized workspace for exam controllers to ensure academic processes are running smoothly."

2. How we have built it

We used the same **Bootstrap** foundation but changed the "Information Architecture":

- **Header/Navbar:** Contains the logo and admin profile settings.
- **Sidebar Navigation:** Links specifically for 'Manage Exams', 'Student List', 'Result Portal', and 'Schedule'.
- **Main Dashboard Content:**
 - Used **Bootstrap Cards** to show high-level metrics (e.g., "Active Exams: 12", "Total Students: 450").
 - Used **Bootstrap Tables** with the `.table-hover` and `.table-striped` classes to display a list of upcoming exams or student grades clearly.
- **Responsiveness:** Used the **Grid System** to ensure that on a tablet, the 'Active Exam' list takes up the full width, while on a desktop, it sits next to a 'Recent Notifications' panel.

3. The Basics

- **Tables in Bootstrap:** To make a table look good, we apply the `.table` class. To make it responsive (scrollable on small screens), we wrap the `<table>` in a `<div>` with the class `.table-responsive`.
- **Typography:** We used Bootstrap classes like `.display-4` for large numbers and `.text-muted` for sub-labels to create **visual hierarchy** (guiding the examiner's eye to important info first).
- **Components vs. Layout:** The "Layout" is the grid (rows and cols), while "Components" are the actual pieces like Buttons, Cards, and Navs.

4. What changes the examiner might ask you to make

- **"Add a 'Delete' button to each row in the table":** You would go to the `<td>` (table data) tag in the last column and add a `<button class="btn btn-danger">Delete</button>`.
- **"Make the table headers dark":** You would add the class `.table-dark` to the `<thead>` tag.
- **"Change the icon next to 'Manage Exams'":** If you used **FontAwesome** or **Bootstrap Icons**, you would change the class name (e.g., from `bi-book` to `bi-pencil`).

5. Usual Viva Questions & Answers

- **Q: What is the purpose of <thead> and <tbody> in a table?**
 - **A:** They provide semantic meaning. <thead> holds the titles of the columns, and <tbody> holds the actual data. It also helps with styling and screen readers.
 - **Q: What is a "Utility Class" in Bootstrap?**
 - **A:** It's a single-purpose class. For example, mt-3 stands for "margin-top: 3 units." These classes allow us to style elements without writing custom CSS.
 - **Q: How do you align text to the center in Bootstrap?**
 - **A:** Simply add the class text-center to the parent element.
 - **Q: What is the difference between an ID and a Class?**
 - **A:** An **ID** is unique and should only be used for one element on a page (like a specific "submit" button). A **Class** can be reused on multiple elements (like all "cards" on a dashboard).
-

Practical 3: Simple Mobile Website

(Focus: jQuery Mobile)

1. What we have made

"Sir/Ma'am, I have developed a **Mobile-Optimized Web Application** using the **jQuery Mobile framework**. Unlike the previous dashboards which were 'responsive' versions of desktop sites, this is a **Touch-First** application. It mimics the look and feel of a native mobile app with features like finger-friendly buttons, a header/footer that stays in place, and smooth transitions between sections."

2. How we have built it

This practical uses a very specific structure called the **Single Page Template**:

- **The Framework:** We linked the jQuery library and the jQuery Mobile CSS/JS files.
- **Data-Roles:** This is the most important part. We used the data-role attribute to tell the browser how to behave.
 - <div data-role="page"> defines a "screen."
 - <div data-role="header"> and <div data-role="footer"> create the top and bottom bars.

- `<div data-role="content">` holds the main text or buttons.
- **Navigation:** We used standard `<a>` (anchor) tags, but because of jQuery Mobile, clicking them creates a "slide" or "fade" transition between different div IDs instead of loading a whole new file.

3. The Basics

- **Touch-Friendly UI:** Mobile users use thumbs, not mouse cursors. jQuery Mobile automatically makes buttons larger and adds "active" states when you tap them.
- **The Single-File Concept:** You can have five different "pages" for your app all inside **one** .html file. Each "page" is just a div with a unique ID.
- **Themes:** jQuery Mobile uses "Swatches" (like Theme A, Theme B). You change the look by simply adding `data-theme="b"`.

4. What changes the examiner might ask you to make

- **"Change the transition effect":** You would go to your link tag and change `data-transition="slide"` to `data-transition="pop"`, `data-transition="fade"`, or `data-transition="flip"`.
- **"Change the color theme":** You would locate the `data-role="page"` or `data-role="header"` and change `data-theme="a"` (usually light) to `data-theme="b"` (usually dark).
- **"Add a back button":** You can add `data-add-back-btn="true"` to the header div.

5. Usual Viva Questions & Answers

- **Q: What is the main difference between Bootstrap and jQuery Mobile?**
 - **A:** Bootstrap is designed to make desktop sites look good on mobile (Responsive). jQuery Mobile is designed to make websites look and behave like native mobile apps (Touch-Optimized).
- **Q: What is the purpose of the data-role attribute?**
 - **A:** It is a custom attribute used by the jQuery Mobile engine to identify the element's function (e.g., page, header, button) and apply the correct styles and behaviors automatically.
- **Q: How do you link two "pages" within the same HTML file?**

- **A:** You give each data-role="page" a unique ID (e.g., id="home" and id="about") and then link them using the hash symbol: Go to About.
 - **Q: What is a "CDN" in the context of jQuery?**
 - **A:** It stands for Content Delivery Network. It allows us to include jQuery libraries by pointing to a URL (like Google or Microsoft's servers) instead of downloading the file locally.
-

To make the **"Single Page"** concept of Practical 3 crystal clear, imagine a physical **notebook**.

Even though you have 10 pages in the notebook, it is still **one single object** in your hand. In Practical 3, your .html file is the notebook, and each div with data-role="page" is a page inside it. When you click a link, jQuery Mobile doesn't go to a new file; it just "flips the page" inside the same file with a nice animation.

Practical 4: User Registration with AJAX and Local Storage

(Focus: JavaScript, AJAX, Web Storage)

1. What we have made

"Sir/Ma'am, I have built a **Dynamic User Registration System**. Unlike the previous practicals which were mostly about design, this one focuses on **Data Handling**. When a user fills out the form and clicks 'Register', the data is captured by JavaScript, saved into the browser's **Local Storage**, and then displayed on a new page or list—all without the page needing to refresh manually."

2. How we have built it

We used three main technologies working together:

- **The Form (HTML):** A standard form with inputs for name, email, etc.
- **The Logic (JavaScript):** We wrote a script that "listens" for the submit button. It prevents the default form submission (which usually refreshes the page) so we can handle the data ourselves.
- **AJAX (Conceptual/Practical):** We used an AJAX POST method simulation to "push" the data into an array or object.

- **Local Storage:** We used `localStorage.setItem()` to save the user data. Since Local Storage only saves strings, we used `JSON.stringify()` to convert our data object into a text format.

3. The Basics

- **What is AJAX?** It stands for **Asynchronous JavaScript and XML**. It allows a web page to request data from a server (or send data) in the background so the user can keep interacting with the page.
- **What is Local Storage?** It is a small database built into your browser. Data saved here **stays there even if you close the browser or restart your computer**.
- **JSON (JavaScript Object Notation):** This is the "language" used to transport data. It looks like this: `{"name": "John", "age": 30}`.

4. What changes the examiner might ask you to make

- **"Clear the data":** They might ask you to add a "Reset" button that clears the stored users. You would use `localStorage.clear()` or `localStorage.removeItem('key')` in your code.
- **"Add an extra field":** "Add a phone number field." You would add an `<input>` in HTML and then update your JavaScript to "grab" that new value using `document.getElementById()`.
- **"Show an alert":** "Show a message saying 'Registration Successful!'" You would add `alert("Success!");` inside your submission function.

5. Usual Viva Questions & Answers

- **Q: Why use `event.preventDefault()` in the submit function?**
 - **A:** By default, HTML forms try to send data to a new URL and refresh the page. This command stops that "default behavior" so our JavaScript can handle the data silently in the background.
- **Q: What is the difference between Local Storage and Session Storage?**
 - **A: Local Storage** lasts forever (until you delete it). **Session Storage** is deleted the moment you close that specific browser tab.
- **Q: Why do we need `JSON.parse()`?**

- **A:** When we get data *out* of Local Storage, it comes as a string (text). `JSON.parse()` turns that text back into a JavaScript Object so we can easily read the name, email, etc.
- **Q: What are the HTTP methods used in AJAX?**
 - **A:** The most common are **GET** (to fetch/read data) and **POST** (to send/save data).

the "**Single Page**" concept of Practical 3 crystal clear, imagine a physical **notebook**.

Even though you have 10 pages in the notebook, it is still **one single object** in your hand. In Practical 3, your .html file is the notebook, and each div with `data-role="page"` is a page inside it. When you click a link, jQuery Mobile doesn't go to a new file; it just "flips the page" inside the same file with a nice animation.

Practical 4: User Registration with AJAX and Local Storage

(Focus: JavaScript, AJAX, Web Storage)

1. What we have made

"Sir/Ma'am, I have built a **Dynamic User Registration System**. Unlike the previous practicals which were mostly about design, this one focuses on **Data Handling**. When a user fills out the form and clicks 'Register', the data is captured by JavaScript, saved into the browser's **Local Storage**, and then displayed on a new page or list—all without the page needing to refresh manually."

2. How we have built it

We used three main technologies working together:

- **The Form (HTML):** A standard form with inputs for name, email, etc.
- **The Logic (JavaScript):** We wrote a script that "listens" for the submit button. It prevents the default form submission (which usually refreshes the page) so we can handle the data ourselves.
- **AJAX (Conceptual/Practical):** We used an AJAX POST method simulation to "push" the data into an array or object.
- **Local Storage:** We used `localStorage.setItem()` to save the user data. Since Local Storage only saves strings, we used `JSON.stringify()` to convert our data object into a text format.

3. The Basics

- **What is AJAX?** It stands for **Asynchronous JavaScript and XML**. It allows a web page to request data from a server (or send data) in the background so the user can keep interacting with the page.
- **What is Local Storage?** It is a small database built into your browser. Data saved here **stays there even if you close the browser or restart your computer**.
- **JSON (JavaScript Object Notation):** This is the "language" used to transport data. It looks like this: `{"name": "John", "age": 30}`.

4. What changes the examiner might ask you to make

- **"Clear the data":** They might ask you to add a "Reset" button that clears the stored users. You would use `localStorage.clear()` or `localStorage.removeItem('key')` in your code.
- **"Add an extra field":** "Add a phone number field." You would add an `<input>` in HTML and then update your JavaScript to "grab" that new value using `document.getElementById()`.
- **"Show an alert":** "Show a message saying 'Registration Successful!'" You would add `alert("Success!");` inside your submission function.

5. Usual Viva Questions & Answers

- **Q: Why use `event.preventDefault()` in the submit function?**
 - **A:** By default, HTML forms try to send data to a new URL and refresh the page. This command stops that "default behavior" so our JavaScript can handle the data silently in the background.
- **Q: What is the difference between Local Storage and Session Storage?**
 - **A: Local Storage** lasts forever (until you delete it). **Session Storage** is deleted the moment you close that specific browser tab.
- **Q: Why do we need `JSON.parse()`?**
 - **A:** When we get data *out* of Local Storage, it comes as a string (text). `JSON.parse()` turns that text back into a JavaScript Object so we can easily read the name, email, etc.
- **Q: What are the HTTP methods used in AJAX?**

- **A:** The most common are **GET** (to fetch/read data) and **POST** (to send/save data).

More for Practical 4: The "Under the Hood" Details

Use this to understand the "Flow of Data"

The "Step-by-Step" Data Journey

1. **Capture:** User types "John" into the input. JavaScript uses `document.getElementById('name').value` to grab that string.
2. **Package:** You put all the fields into a JavaScript **Object** (e.g., `let user = {name: "John", email: "john@mail.com"}`).
3. **Serialize:** Local Storage cannot store Objects; it only stores **Strings**. So, you use `JSON.stringify(user)` to turn that object into a long piece of text.
4. **Storage:** You use `localStorage.setItem('userList', textData)`.
5. **Retrieval:** To show it on a new page, you pull it out using `getItem`, turn it back into an object using `JSON.parse()`, and use a for loop to create `<li>` or `<tr>` tags to show it to the user.

Key Term: "State Management"

The examiner might ask: *"How do you keep track of the data?"* **Your Answer:** "I am using **Client-Side State Management**. Instead of sending the data to a server database immediately, I am managing the 'state' (the current list of users) directly in the user's browser memory."

Practicals 5, 6, & 7: The "Themed" Dashboards

(5: College, 6: Healthcare, 7: Ecommerce - repeated or modified)

1. What we have made

"Sir/Ma'am, these practicals demonstrate how the **same technical framework (Bootstrap)** can be applied to different industries by simply changing the **Components** and **Visual Language**."

- **Practical 5 (College):** Focuses on Student/Faculty management.
- **Practical 6 (Healthcare):** Focuses on Patient/Doctor stats and appointment scheduling.

- **Practical 7 (Ecommerce):** Focuses on Sales, Inventory, and Revenue tracking.

2. How we have build (The "Repeatable Formula")

Regardless of the theme, the "Skeleton" remains exactly the same:

- **Navigation:** A top bar for search and notifications.
- **Sidebar:** Links to the specific modules (e.g., "Wards/Beds" for Healthcare vs. "Classrooms" for College).
- **The Grid:** Usually a row containing four col-md-3 divs for the top "Quick Stats."
- **The Content:** A large col-md-8 for a data table and a col-md-4 for a "Recent Activity" feed or a small calendar.

3. Basics: Semantic Differences

- **Colors:** In **Healthcare (Prac 6)**, we use "clean" or "urgent" colors like bg-info (light blue) or bg-danger (red for emergency alerts). In **College (Prac 5)**, we might use more "professional" colors like bg-primary (deep blue) or bg-secondary.
- **Icons:** We swap icons based on the context. A bi-person-heart for a doctor vs. a bi-mortarboard (graduation cap) for a college.

4. What changes can the examiner ask?

- **"Make this Healthcare dashboard look like a Dark Mode app":** You would change classes like bg-light to bg-dark and text-dark to text-white.
- **"Swap the 'Total Patients' card with the 'Appointments' card":** They are testing if you understand how to move div blocks around in your HTML code.
- **"Add a 'Status' badge to the table":** You would use the Bootstrap **Badge** component: `<span class="badge bg-success">Discharged</span>`.

5. Usual Viva Questions & Answers

- **Q: Why is the 'Grid System' better than using HTML Tables for the whole layout?**
 - **A:** HTML Tables are not naturally responsive. If you use a table for layout, it will break on a phone. The Grid System is designed to "reflow" and stack elements perfectly.
- **Q: What is a 'Card' in Bootstrap?**

- **A:** It is a flexible and extensible content container. It includes options for headers, footers, and a wide variety of content. It is the standard way to display "blocks" of info on a dashboard.
- **Q: If I want to add a space between two cards, what class do I use?**
 - **A:** You can use **Margin** utility classes like m-3 (margin on all sides) or me-2 (margin-end / right side).
- **Q: How do you fix a Sidebar so it doesn't disappear when you scroll?**
 - **A:** Use the Bootstrap class .sticky-top or the custom CSS property position: fixed;.

Practical 8: Version Control with Git and GitHub

(Focus: DevOps, Collaboration, History Tracking)

1. What we have made

"Sir/Ma'am, for this practical, I haven't just made a 'page,' but a **Workflow**. I have established a **Version Control System** for my project. I created a local repository to track every change I make to my code, and then I connected it to a remote repository on **GitHub**. This allows me to save 'snapshots' of my work, collaborate with others, and ensure that if I make a mistake, I can always roll back to a previous working version."

2. How we have built it

We followed the standard Git lifecycle commands:

1. **Initialize:** We used git init to turn our project folder into a Git repository.
2. **Staging:** We used git add . to tell Git which files we want to include in our next "snapshot."
3. **Commit:** We used git commit -m "message" to permanently save those changes to our local history.
4. **Connect:** We used git remote add origin [URL] to link our local folder to our GitHub account.
5. **Push:** We used git push -u origin main to upload our local code to the cloud.

3. The Basics

- **What is Git?** It is a **distributed version control system**. "Distributed" means every developer has a full copy of the project history on their own machine.
- **The Repository (Repo):** Think of this as a "Smart Folder" that remembers every version of every file inside it.
- **Commit:** This is like a "Save Point" in a video game. It records exactly what the code looked like at that specific moment.
- **Branching:** This allows you to "branch off" from the main code to test a new feature without breaking the original project.

4. What changes the examiner might ask you to make

- **"Show me your commit history":** You would type `git log` in the terminal. This shows a list of all your previous "saves."
- **"Check the current status":** You would type `git status`. It will show you which files are modified but not yet "added" or "committed."
- **"Create a new branch":** You would type `git branch feature-name` and then `git checkout feature-name`.
- **"Fix a typo and push it":** You change the text in your HTML, then run the sequence: `git add . -> git commit -m "fixed typo" -> git push`.

5. Usual Viva Questions & Answers

- **Q: What is the difference between Git and GitHub?**
 - **A:** **Git** is the local tool (software) you use on your computer to track changes. **GitHub** is the website (service) that hosts your Git repositories in the cloud.
- **Q: What does the .gitignore file do?**
 - **A:** It tells Git which files or folders to **ignore** (not track). Usually, we ignore large folders like `node_modules` or sensitive files like passwords.
- **Q: What is a 'Pull Request' (PR)?**
 - **A:** On GitHub, a Pull Request is a way to tell others that you have pushed changes to a branch and want to merge them into the main project. It's a request for a "code review."
- **Q: What is the difference between git clone and git pull?**

- **A:** git clone is used the **first time** to download an entire project. git pull is used to get the **latest updates** for a project you already have.

Practical 9: Create Docker Container Environment

(Focus: Virtualization, DevOps, Environment Isolation)

1. What we have made

"Sir/Ma'am, I have created a **Dockerized Environment** for my application. Instead of just running the code directly on my laptop, I have 'packaged' the application into a **Container**. This container includes everything the app needs to run: the code, the runtime (like Node.js), system tools, and libraries. This ensures the 'It works on my machine' problem is solved—if it works in this Docker container, it will work on any computer in the world."

2. How we have built it

We used a tool called **Docker** and followed these steps:

1. **The Blueprint (Dockerfile):** We wrote a text file named Dockerfile. This file contains a list of instructions (like a recipe) on how to build the environment.
2. **The Image:** We ran a command (docker build) that took the Dockerfile and turned it into an **Image**—a lightweight, standalone, executable package.
3. **The Container:** We ran the image (docker run) to start the **Container**. This is the actual "living" process where the app is running.
4. **NVIDIA Docker (Optional/Specific):** If we used NVIDIA Docker, it means we allowed the container to access the physical Graphics Card (GPU) of the computer for high-performance tasks.

3. The Basics

- **What is a Container?** It's like a "bubble" around your app. Inside the bubble, the app thinks it has its own operating system, but it's actually sharing the main computer's resources very efficiently.
- **Dockerfile Keywords:**
 - FROM: Tells Docker which base image to start with (e.g., FROM node:18).
 - COPY: Moves your code from your pendrive into the container.

- RUN: Executes commands (like npm install) inside the container.
- CMD: The final command that actually starts your app (e.g., node server.js).

4. What changes the examiner might ask you to make

- **"Show me your Dockerfile":** Open the file and explain the FROM and COPY lines.
- **"List your running containers":** You would type docker ps in the terminal.
- **"Stop the container":** You would type docker stop [container_id].
- **"Change the port":** If your app runs on port 3000, they might ask you to map it to port 8080 on your laptop. You would change the "publish" flag to -p 8080:3000.

5. Usual Viva Questions & Answers

- **Q: What is the difference between a Virtual Machine (VM) and a Docker Container?**
 - **A:** A **VM** includes a full "Guest Operating System," making it very heavy and slow. A **Container** shares the "Host Operating System's Kernel," making it much smaller, faster, and more efficient.
- **Q: What is an 'Image' versus a 'Container'?**
 - **A:** An **Image** is the "class" or "blueprint" (stored on disk). A **Container** is the "object" or "instance" (running in memory).
- **Q: Why use NVIDIA Docker?**
 - **A:** Standard Docker containers cannot "see" the GPU hardware. NVIDIA Docker provides the drivers and tools so the container can use the GPU for things like Machine Learning or Data Analysis.
- **Q: What is Docker Hub?**
 - **A:** It is a cloud-based library (like GitHub but for Docker) where people share pre-built images (like official images for Node.js, Python, or MongoDB).

Practical 10: Create an Angular Application

(Focus: Single Page Applications (SPA), TypeScript, Component-Based Architecture)

1. What we have made

"Sir/Ma'am, I have developed a **Single Page Application (SPA)** using **Angular**. Unlike the previous dashboards made with plain HTML/CSS, this is a highly structured application where the page never reloads. I created two main modules: a **User Registration** page and a **Dashboard**. The app uses 'Routing' to switch between these views instantly, providing a smooth, app-like experience for the user."

2. How we have built it

We used the **Angular CLI** (Command Line Interface) and the following concepts:

- **Components:** We broke the app into pieces. The register-component handles the form, and the dashboard-component displays the data.
- **TypeScript:** Instead of standard JavaScript, we used TypeScript. This allows us to use "classes" and "types," making the code more organized and easier to debug.
- **Routing:** We configured an AppRoutingModuleModule so that when the URL changes to /dashboard, Angular knows to swap the Registration HTML with the Dashboard HTML without a page refresh.
- **Data Binding:** We used **Two-Way Data Binding** (`[(ngModel)]`). This means if a user types in the input box, the variable in our TypeScript code updates automatically in real-time.

3. The Basics

- **SPA (Single Page Application):** A website that interacts with the user by dynamically rewriting the current web page rather than loading entire new pages from a server.
- **Decorators:** You will see @Component in the code. This is a "decorator" that tells Angular that a specific class is a component and links it to its HTML and CSS files.
- **Directives:** Special markers like *ngIf (to show/hide elements) or *ngFor (to repeat elements in a list).

4. What changes the examiner might ask you to make

- **"Show me where the routes are defined":** You would open app-routing.module.ts and point to the routes array.
- **"Add a new link to the menu":** You would go to the HTML of your navigation component and add an <a> tag with a routerLink="/new-path" attribute.

- **"Change the title on the Dashboard":** You would go to `dashboard.component.ts`, change the value of a variable (e.g., `title = 'Welcome User'`), and show how it automatically updates the HTML.

5. Usual Viva Questions & Answers

- **Q: What is the purpose of the `Node_modules` folder in an Angular project?**
 - **A:** It contains all the external libraries and dependencies that Angular needs to run. (Note: We usually don't copy this to a pendrive if it's too big; we run `npm install` to recreate it).
- **Q: What is the difference between a Component and a Module in Angular?**
 - **A:** A **Component** controls a small patch of the screen (the UI). A **Module** is a big container (like a box) that groups related components, services, and routing together.
- **Q: What is 'Dependency Injection' (DI)?**
 - **A:** It is a design pattern where a class receives its dependencies from an external source rather than creating them itself. In Angular, we use it to give components access to "Services" (like data fetching).
- **Q: What command do you use to run the Angular app?**
 - **A:** `ng serve`. This starts a local development server, usually at `localhost:4200`.

Practical 11: Node.JS Application Serving a Static Website

(Focus: Server-side JavaScript, File System, HTTP Basics)

1. What we have made (Detailed Explanation)

"Sir/Ma'am, I have built a Custom Web Server using Node.js. In a normal scenario, a browser just opens a file from your hard drive. But here, I have created an active Backend Service.

This application stays 'awake' on the computer, listening for incoming network requests. When a user enters the server's URL, the Node.js logic intercepts that request, searches my project folder for the requested file (like `index.html`), reads the raw data, and sends it back through the 'pipe' of the internet (the HTTP protocol) so the browser can display it. This is exactly how professional websites like Google or Facebook serve their initial pages."

2. How we have built it (The Architecture)

We used a modular approach:

- Initialization: We ran `npm init` to create the `package.json`, which acts as the 'manual' for our server.
- The Framework (Express.js): While Node.js has a built-in `http` module, we used Express.js because it simplifies "Routing."
- Static Middleware: We used a special command—`app.use(express.static('public'))`. This is a powerful line of code that tells the server: *"If someone asks for a file, look inside the 'public' folder and give it to them automatically."*
- The Listener: We defined a Port (like a specific door on the computer, e.g., 3000) using `app.listen()`. This keeps the script running indefinitely.

3. The Basics (The "Why" behind the "What")

- The Request-Response Cycle: This is the most important concept. The Request is the user asking for a page; the Response is the server sending the data back.
- Middleware: Think of middleware as a "security guard" or "translator" that sits between the request and the response. In this practical, the middleware's job is to find the right file.
- Common File Extensions:
 - `*.js`: Your server logic.
 - `.json`: Your configuration and metadata.
 - `.html/.css`: Your static assets.

4. What changes can the examiner ask?

- "Show me how to stop and restart the server": They want to see you press `Ctrl + C` in the terminal to stop it and type `node server.js` (or `nodemon`) to start it again.
- "What happens if I change the filename from `index.html` to `home.html`?: You would explain that the server might not find it automatically unless you update the code to point to the new filename.
- "Can you host images this way?": Answer: "Yes, as long as the image is in the 'static' folder, the server will treat it just like a CSS or HTML file."

- "How do I see this on another computer?": Answer: "If both computers are on the same Wi-Fi, the other person can type my computer's IP address followed by the port (e.g., 192.168.1.5:3000)."

5. Usual Viva Questions & Answers

- **Q: What is the difference between a 'Static' and 'Dynamic' website?**
 - **A:** A Static website (like this practical) stays the same for every user. A Dynamic website (like Facebook) changes its content based on who is logged in or what is in the database.
- **Q: What is npm and why is the node_modules folder so large?**
 - **A:** npm is the tool that downloads code written by other people so we don't have to "reinvent the wheel." The node_modules folder is large because it contains all the code for Express, its dependencies, and their dependencies.
- **Q: What is a 'Port' in networking?**
 - **A:** A port is a communication endpoint. A computer has thousands of ports. Port 80 is for standard web traffic, but developers usually use 3000 or 8080 to avoid interfering with system services.
- **Q: Why do we use `const path = require('path')`?**
 - **A:** Different operating systems use different slashes for folders (Windows uses \, Linux uses /). The path module ensures our server works on both by handling these slashes automatically.

Practical 12: API for Create & Update Operations

(Focus: Node.js, Express, MongoDB, and Mongoose)

1. What we have made (The "Brain and Memory")

"Sir/Ma'am, in this practical, I have moved beyond serving static files to building a **Functional Backend API**. I have connected my Node.js server to a **MongoDB database**.

Specifically, I implemented two major 'write' capabilities:

- **Create:** Allowing the application to take new user data and save it permanently in the database.
- **Update:** Allowing the application to find an existing record and modify its details.

This is the foundation of any application that needs to 'remember' its users or their preferences."

2. How we have built it (The Connection Logic)

We used a **Three-Layer Architecture**:

1. **The Database (MongoDB):** A NoSQL database where data is stored in 'Documents' (which look like JSON).
2. **The ODM (Mongoose):** We used the **Mongoose** library. It acts as a bridge. We defined a **Schema** (a blueprint) so the database knows exactly what a 'User' should look like (e.g., Name must be a string, Age must be a number).
3. **The Routes (Express):**
 - **POST Route:** We created an endpoint (e.g., /api/users) to **Create** data.
 - **PUT/PATCH Route:** We created an endpoint (e.g., /api/users/:id) to **Update** specific data using a unique ID.

3. The Basics (The "CRUD" Concept)

- **What is an API?** It stands for **Application Programming Interface**. It's like a waiter in a restaurant: you (the frontend) give the waiter an order (the request), the waiter takes it to the kitchen (the database), and brings back your food (the response).
- **JSON Body:** When we "Create" or "Update," we send data in the **Request Body**. This is a JSON object sent from the client to the server.
- **Asynchronous Operations (async/await):** Talking to a database takes time. We use `async` and `await` keywords to tell JavaScript to "wait" for the database to finish saving before moving to the next line of code.

4. What changes can the examiner ask?

- **"Show me your Schema":** They want you to open your models file and explain how you defined the fields (e.g., `required: true`).
- **"Update a different field":** "Instead of updating the name, update the email address." You would show how you target the specific key in the update function.
- **"How do you know if the data was saved?": Answer:** "I use `console.log` to see the result in the terminal, and I can also check **MongoDB Compass** (the visual tool) to see the new record in the collection."

- **"What is the difference between PUT and PATCH?": Answer:** "PUT usually replaces the entire document, while PATCH is used for partial updates (only changing one or two fields)."

5. Usual Viva Questions & Answers

- **Q: What is Mongoose?**
 - **A:** Mongoose is an **Object Data Modeling (ODM)** library for MongoDB and Node.js. It manages relationships between data, provides schema validation, and is used to translate between objects in code and the representation of those objects in MongoDB.
- **Q: Why use MongoDB instead of a traditional SQL database?**
 - **A:** MongoDB is **flexible**. It uses a "Schema-less" design, meaning we can change the data structure easily without complex migrations. It's also very fast for handling large amounts of unstructured data.
- **Q: What does `body-parser` or `app.use(express.json())` do?**
 - **A:** It is middleware that tells the server to "read" the JSON data coming in from the request so we can use it in our code as `req.body`.
- **Q: What is a 'Collection' in MongoDB?**
 - **A:** In SQL, you have 'Tables'. In MongoDB, the equivalent is a '**Collection**'. It is a group of related documents (like a collection of Users or a collection of Products).

Practical 13: API for Create & Read Operations

(Focus: Node.js, Express, MongoDB, and Data Retrieval)

1. What we have made (The "Display Logic")

"Sir/Ma'am, in this practical, I have implemented the **Read** capability of the application. While Practical 12 focused on saving data, this one focuses on **Retrieving and Displaying** that data.

I have created two types of Read operations:

- **Read All (Get All Users):** Fetching the entire list of records from the database to show on a dashboard.

- **Read One (Get User by ID):** Searching for a specific individual record based on a unique identifier (ID)."

2. How we have built it (The Fetching Logic)

We used **Express GET routes** and **Mongoose Query Methods**:

- **The Route:** We defined a GET endpoint (e.g., `app.get('/api/users', ...)`).
- **The Query:** We used `User.find()` to talk to MongoDB. This command tells the database: "Give me every document inside the 'Users' collection."
- **The Logic:**
 1. The client sends a **GET request**.
 2. The server executes the **Mongoose query**.
 3. The server waits for the database (using `await`).
 4. The server sends the data back to the client in **JSON format**.

3. The Basics (The "Fetch" Concept)

- **What is a GET Request?** It is the standard HTTP method used to **request data** from a specified resource. Unlike POST, a GET request does not change the state of the server or database.
- **Querying:** This is the act of asking the database a specific question (e.g., "Find all users where age > 18").
- **JSON Array:** When we "Read All," the server sends back an **Array of Objects** [...]. When we "Read One," it sends back a **Single Object** {...}.

4. What changes can the examiner ask?

- **"Find a user by their Name instead of ID":** They want to see if you can change the query from `findById(id)` to `findOne({ name: 'John' })`.
- **"Filter the results":** "Show only users who are 'Admin'." You would update your query to `User.find({ role: 'Admin' })`.
- **"Sort the data":** "Show the newest users first." You would add a sort method: `User.find().sort({ createdAt: -1 })`.

- **"How do you test this API without a frontend?": Answer:** "I use tools like **Postman** or the **Thunder Client** extension in VS Code to send GET requests and see the JSON response."

5. Usual Viva Questions & Answers

- **Q: What happens if the database is empty when you call find()?**
 - **A:** MongoDB will return an **empty array []**. It is not an error; it just means no records match the criteria.
- **Q: What is the difference between find() and findOne()?**
 - **A:** find() returns an **array** containing all matching documents. findOne() returns only the **first** document that matches the criteria (or null if none are found).
- **Q: What is a 'Route Parameter' (e.g., /:id)?**
 - **A:** It is a variable part of the URL. If the user visits /api/users/123, the :id tells Express to capture "123" so we can use it to search the database.
- **Q: Why do we use a try...catch block in the backend code?**
 - **A:** To handle **Errors**. For example, if the database connection fails, the catch block prevents the whole server from crashing and sends a "500 Internal Server Error" message to the user instead.

I'll take that as an enthusiastic "Yes!" to finish off the series. Let's wrap up the final practical for your exam.

Practical 14: API for Create & Delete Operations

(Focus: Node.js, Express, MongoDB, and Resource Removal)

1. What we have made (The "Cleanup Logic")

"Sir/Ma'am, in this final practical, I have implemented the **Delete** functionality. This completes the lifecycle of data management. I built an endpoint that allows the system to permanently remove a record from the MongoDB collection. This is critical for features like 'Delete Account,' 'Remove Item from Cart,' or 'Clear History'."

2. How we have built it (The Removal Logic)

We used the **DELETE** HTTP method and targeted specific IDs:

- **The Route:** We defined a DELETE endpoint (e.g., `app.delete('/api/users/:id', ...)`).
- **The Identifier:** We use the unique **ObjectID** that MongoDB automatically assigns to every document to make sure we don't delete the wrong person by mistake.
- **The Logic:**
 1. The client sends a **DELETE request** with an ID in the URL.
 2. The server uses the `findByIdAndDelete(id)` Mongoose method.
 3. The database searches for that specific ID and removes it.
 4. The server sends back a confirmation message (e.g., "User successfully deleted").

3. The Basics (The "Destructive" Operation)

- **What is a DELETE Request?** It is an idempotent HTTP method, meaning once a resource is deleted, calling the same delete request again will not change anything (since the resource is already gone).
- **Security Note:** In a real-world app, you wouldn't let just *anyone* delete data; you would check for "Admin" permissions first.
- **Params:** We get the ID from `req.params.id`. This is the part of the URL that changes.

4. What changes can the examiner ask?

- **"Delete all users at once":** They are testing if you know the "nuclear option." You would use `User.deleteMany({})`. **Be careful with this!**
- **"Don't delete, just 'Deactivate'":** This is called a **Soft Delete**. You would explain that instead of removing the record, you would just change a field like `isActive: false` using an **Update** operation.
- **"Check if the user exists before deleting":** You could add logic to first find the user, and if they aren't there, return a "404 Not Found" error.

5. Usual Viva Questions & Answers

- **Q: What is an `_id` in MongoDB?**

- **A:** It is a unique 12-byte identifier (Primary Key) automatically created for every document. It ensures that even if two users have the exact same name and age, they can still be distinguished and deleted separately.
 - **Q: What is the HTTP status code for a successful deletion?**
 - **A:** Usually **200 OK** (with a message) or **204 No Content** (if you don't need to send a message back).
 - **Q: Can you recover a document once it is deleted using findByIdAndDelete?**
 - **A:** No. Once it is removed from the database, it is gone permanently unless you have a database backup.
 - **Q: What is the difference between deleteOne() and deleteMany()?**
 - **A:** deleteOne() removes the **first** document that matches the criteria. deleteMany() removes **all** documents that match (e.g., delete all users from "Pune").
-

CC1: Transfer File from VM to VM

(Focus: Virtual Networking and CLI Proficiency)

1. What we have made (In-depth Explanation)

"Sir/Ma'am, for this practical, I managed **Infrastructure-as-a-Service (IaaS)**. I configured two Virtual Machines (VMs)—Ubuntu and Kali Linux—to exist on the same **Virtual Local Area Network (VLAN)** using a NAT Network. By using the **Secure Copy Protocol (SCP)**, I demonstrated that data can be moved securely between isolated cloud instances using encrypted 'tunnels'."

2. How we have built it (The Steps)

- **Hypervisor Configuration:** Used Oracle VM VirtualBox to simulate hardware distribution between two separate operating systems.
- **NAT Networking:** Configured a "NAT Network" in Global Preferences, which acts as a virtual router providing a private IP range (e.g., 10.0.2.x).
- **Service Activation:** Installed net-tools to find the IP address and openssh-server on the destination machine (Ubuntu) to enable the file transfer protocol.

- **Command Execution:** On the source machine (Kali), I used the scp command to send the file directly to the recipient's IP.

3. "What If" Troubleshooting Scenarios

- **Scenario: "The command fails with 'Connection Refused'."**
 - **Response:** "This usually means the SSH service isn't running on the destination VM. I would verify it by running `sudo service ssh status` and start it with `sudo service ssh start`."
- **Scenario: "The machines can't see each other (Ping fails)."**
 - **Response:** "I would check the **VirtualBox Network Settings** for both VMs to ensure they are both attached to the *same* named NAT Network. If they are on different networks, they cannot communicate."
- **Scenario: "How do you transfer a whole directory instead of one file?"**
 - **Response:** "I would add the -r (recursive) flag to the command: `scp -r MyFolder/ username@IP_Address:destination_path`."

CC2: Custom Application on Salesforce Cloud

(Focus: SaaS and Low-Code Development)

1. What we have made (In-depth Explanation)

"Sir/Ma'am, I developed a **Student Details Manager** application using **Software-as-a-Service (SaaS)** principles. I acted as a System Administrator to customize a multi-tenant cloud environment on the **Salesforce Lightning platform**, building a database and a mobile-ready interface without traditional backend coding."

2. How we have built it (The Steps)

- **Object Definition:** Created a "Custom Object" called `Student_Detail`, which serves as our database table.
- **Field Engineering:** Added specific fields like **Email**, **Phone**, and **Date of Birth** to define what data we collect.
- **App Packaging:** Used the **App Manager** to bundle our custom tab and object into a standalone "Lightning App".

- **Security Mapping:** Assigned the app specifically to the **System Administrator** profile to control who has access to this data.

3. "What If" Troubleshooting Scenarios

- **Scenario: "The app is created but you can't find it in the menu."**
 - **Response:** "I would check if I created a **Custom Tab** for the object. Without a Tab, the object exists in the database but is invisible in the user interface."
- **Scenario: "A user tries to save a student record without an email address."**
 - **Response:** "If I marked the Email field as '**Required**' during setup, Salesforce will automatically block the save and alert the user to fill the field."
- **Scenario: "The examiner asks you to view the app on a mobile device."**
 - **Response:** "Since Salesforce is cloud-native, I can use the **Chrome Developer Tools** to toggle the mobile view (device toolbar) and demonstrate the responsive Lightning UI."